



Модуль 12. Кейсы и финальный проект

Введение в модуль

Все одиннадцать предыдущих модулей — это инструменты. Мы изучили: итераторы и генераторы как фундамент `async / await` (Модуль 1); `event loop`, корутины и синхронизацию (Модуль 2); сетевые протоколы от TCP до HTTP/3 (Модуль 3); FastAPI, Pydantic V2, middleware и lifespan (Модуль 4); Dependency Injection и тестируемость (Модуль 5); асинхронные базы данных и миграции (Модуль 6); криптографию, JWT и безопасность (Модуль 7); тестирование от unit до нагрузочного (Модуль 8); интеграцию ML: inference, батчинг, streaming, Feature Store (Модуль 9); продакшн: Docker, observability, масштабирование, CI/CD (Модуль 10); архитектурные паттерны: чистая архитектура, Circuit Breaker, Event-Driven, CAP-теорема (Модуль 11).

Этот модуль — **синтез**. Мы разберём три реальных кейса, каждый из которых затрагивает уникальное сочетание технологий, и спроектируем финальный проект, объединяющий всё в единую систему.

12.1. Кейс 1: ML API для real-time классификации изображений

12.1.1. Постановка задачи и архитектура пайплайна

Задача: сервис, который принимает HTTP-запрос с изображением, классифицирует его (например, «кошка» / «собака» / «птица») и возвращает класс с уверенностью. Требование: latency < 200 мс на p95, throughput > 500 RPS.

Полный пайплайн:



12.1.2. Загрузка и валидация изображений

FastAPI предоставляет `UploadFile` для потоковой загрузки файлов. Ключевое слово здесь — **потоковая**: файл не загружается целиком в память, если размер позволяет, а читается по частям.

```
In [ ]: from fastapi import FastAPI, File, UploadFile, HTTPException
        from pydantic import BaseModel
        import imghdr

        app = FastAPI()

        # Белый список MIME-типов
        ALLOWED_TYPES = {"image/jpeg", "image/png", "image/webp"}
        MAX_FILE_SIZE = 10 * 1024 * 1024 # 10 МБ

        class ClassificationResponse(BaseModel):
            class_name: str
            confidence: float
            inference_time_ms: float

        @app.post("/classify", response_model=ClassificationResponse)
        async def classify_image(file: UploadFile = File(...)):
            # 1. Проверка MIME-типа
            if file.content_type not in ALLOWED_TYPES:
                raise HTTPException(400, f"Unsupported type: {file.content_type}")

            # 2. Чтение и проверка размера
            contents = await file.read()
            if len(contents) > MAX_FILE_SIZE:
                raise HTTPException(413, "File too large")

            # 3. Валидация, что это действительно изображение
            # imghdr.what проверяет magic bytes файла
            img_type = imghdr.what(None, contents)
            if img_type not in {"jpeg", "png", "webp"}:
                raise HTTPException(400, "Invalid image content")

            # Дальше – предобработка и inference
            ...
```

Почему `imghdr.what`, а не только `content_type`? `content_type` приходит от клиента и может быть подделан. `imghdr` анализирует **magic bytes** — сигнатуру файла (например, `FF D8 FF` для JPEG). Это защита от атаки «загрузить PHP-скрипт с расширением .jpg».

12.1.3. Предобработка: от сырых байтов к тензору

Модель ожидает вход фиксированного размера (например, 224×224 пикселя, нормализованные по ImageNet). Предобработка — CPU-bound операция (NumPy/OpenCV), которую нельзя выполнять в event loop.

```
In [ ]: import numpy as np
        from PIL import Image
        import io
        from concurrent.futures import ThreadPoolExecutor
        import asyncio

        # Пул потоков для CPU-bound предобработки
        _preprocess_executor = ThreadPoolExecutor(max_workers=4)

        # Параметры модели
        INPUT_SIZE = (224, 224)
        IMAGENET_MEAN = np.array([0.485, 0.456, 0.406])
        IMAGENET_STD = np.array([0.229, 0.224, 0.225])

        def _preprocess_sync(image_bytes: bytes) -> np.ndarray:
            """Синхронная предобработка в пуле потоков."""
            # Загрузка изображения
            image = Image.open(io.BytesIO(image_bytes)).convert("RGB")

            # Resize с сохранением aspect ratio + center crop
            # (упрощённо – прямой resize)
            image = image.resize(INPUT_SIZE, Image.Resampling.LANCZOS)

            # В numpy: [H, W, C] -> [C, H, W]
            arr = np.array(image).astype(np.float32) / 255.0
            arr = (arr - IMAGENET_MEAN) / IMAGENET_STD
            arr = np.transpose(arr, (2, 0, 1)) # HWC -> CHW

            # Добавляем batch dimension: [1, C, H, W]
            return np.expand_dims(arr, axis=0)

        async def preprocess(image_bytes: bytes) -> np.ndarray:
            loop = asyncio.get_running_loop()
            return await loop.run_in_executor(_preprocess_executor, _preprocess_s
```

Почему LANCZOS ? При уменьшении изображения метод интерполяции влияет на качество. LANCZOS (также известен как ANTIALIAS) использует sinc-функцию с окном Ланцоша, минимизирующую артефакты (aliasing) при ресайзе. Для ML это критично: билинейная интерполяция может размыть границы, ухудшая точность классификации.

12.1.4. ONNX Runtime: оптимизированный inference

```
In [ ]: import onnxruntime as ort
import time

# Загрузка сессии один раз при старте (lifespan, Module 4)
_session = None

def init_model(model_path: str):
    global _session
    # Автовыбор провайдера: CUDA -> DirectML -> CPU
    providers = ort.get_available_providers()
    _session = ort.InferenceSession(model_path, providers=providers)
    print(f"Model loaded. Providers: {providers}")

async def predict(image_tensor: np.ndarray) -> tuple[str, float]:
    # ONNX Runtime – синхронный вызов C++ бэкенда, который освобождает GI
    # Но тяжёлый inference всё равно лучше вынести в ThreadPoolExecutor
    loop = asyncio.get_running_loop()

    start = time.perf_counter()
    result = await loop.run_in_executor(
        None, # default executor
        lambda: _session.run(None, {_session.get_inputs()[0].name: image_
    })
    inference_time = (time.perf_counter() - start) * 1000

    # Постобработка: softmax + argmax
    logits = result[0][0] # [num_classes]
    probabilities = softmax(logits)
    class_idx = int(np.argmax(probabilities))
    confidence = float(probabilities[class_idx])

    class_names = ["cat", "dog", "bird"] # загружается из конфига
    return class_names[class_idx], confidence, inference_time

def softmax(x: np.ndarray) -> np.ndarray:
    exp_x = np.exp(x - np.max(x)) # numerical stability
    return exp_x / np.sum(exp_x)
```

Почему `np.exp(x - np.max(x))` ? Softmax: $\sigma(x)_i = \frac{e^{x_i}}{\sum_j e^{x_j}}$. Если x_i большие (сотни), e^{x_i} переполняет float (inf). Вычитание $\max(x)$ до экспоненты не меняет результат (константа в числителе и знаменателе сокращается), но предотвращает переполнение.

12.1.5. Кэширование результатов

Если один и тот же пользователь загружает одно и то же изображение

несколько раз (или разные пользователи загружают популярные изображения), можно кэшировать результат.

```
In [ ]: import hashlib
import json
from redis.asyncio import Redis

async def classify_with_cache(
    image_bytes: bytes,
    redis: Redis,
) -> ClassificationResponse:
    # Хеш изображения как ключ кэша
    image_hash = hashlib.sha256(image_bytes).hexdigest()
    cache_key = f"classify:{image_hash}"

    # Пробуем кэш
    cached = await redis.get(cache_key)
    if cached:
        data = json.loads(cached)
        return ClassificationResponse(**data)

    # Предобработка + inference
    tensor = await preprocess(image_bytes)
    class_name, confidence, inference_time = await predict(tensor)

    response = ClassificationResponse(
        class_name=class_name,
        confidence=confidence,
        inference_time_ms=inference_time,
    )

    # Кладём в кэш с TTL (например, 1 час)
    # Кэшируем только высокоуверенные предсказания (> 0.9)
    if confidence > 0.9:
        await redis.setex(cache_key, 3600, response.model_dump_json())

    return response
```

Почему SHA-256? MD5 и SHA-1 уязвимы к коллизиям. Для кэша это не критично (злоумышленник не атакует кэш), но SHA-256 — стандарт безопасности. Важнее: хеширование 10 МБ изображения занимает < 1 мс, а экономит 100+ мс inference.

12.1.6. Rate limiting и защита от перегрузки

```
In [ ]: from fastapi import Request, HTTPException
import time

# Token Bucket (Module 7, углублённая реализация)
class TokenBucket:
```

```

def __init__(self, capacity: int, refill_rate: float):
    self.capacity = capacity
    self.tokens = float(capacity)
    self.refill_rate = refill_rate # tokens per second
    self.last_refill = time.monotonic()
    self._lock = asyncio.Lock()

    async def consume(self, tokens: int = 1) -> bool:
        async with self._lock:
            now = time.monotonic()
            elapsed = now - self.last_refill
            self.tokens = min(self.capacity, self.tokens + elapsed * self.refill_rate)
            self.last_refill = now

            if self.tokens >= tokens:
                self.tokens -= tokens
                return True
            return False

# Глобальный bucket на endpoint
_classify_bucket = TokenBucket(capacity=100, refill_rate=50) # 100 burst

@app.post("/classify")
async def classify_image(
    request: Request,
    file: UploadFile = File(...),
    redis: Redis = Depends(get_redis),
):
    # Rate limit no IP
    client_ip = request.client.host
    ip_bucket = TokenBucket(capacity=10, refill_rate=2)

    if not await ip_bucket.consume():
        raise HTTPException(429, "Too many requests")

    if not await _classify_bucket.consume():
        raise HTTPException(503, "Server overloaded, try later")

    # Основная логика
    contents = await file.read()
    return await classify_with_cache(contents, redis)

```

Token Bucket vs Leaky Bucket: Token Bucket позволяет **burst** (всплеск) до `capacity`, затем ограничивает `refill_rate`. Это хорошо для ML API: пользователь может отправить 5 изображений сразу, но затем ограничивается. Leaky Bucket выравнивает трафик плавно, но не допускает burst.

12.1.7. Математическая подоплека: метрики классификации

Модель возвращает не только класс, но и уверенность. Как оценить качество?

Confusion Matrix для K классов — матрица $C \in \mathbb{N}^{K \times K}$, где C_{ij} — число примеров класса i , предсказанных как класс j .

Для бинарной классификации (кошка / не-кошка):

	Pred: Да	Pred: Нет
Real: Да	TP	FN
Real: Нет	FP	TN

Метрики:

$\text{Precision} = \frac{TP}{TP + FP}$ (из всех предсказанных «кошек» сколько настоящих?)
 $\text{Recall} = \frac{TP}{TP + FN}$ (из всех настоящих «кошек» сколько нашли?)
 $\text{F1} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$

ROC-AUC: площадь под кривой (False Positive Rate, True Positive Rate).
 $\text{AUC} = 0.5$ — случайное угадывание, $\text{AUC} = 1.0$ — идеальное разделение.

$\text{TPR} = \frac{TP}{TP + FN}$, $\text{FPR} = \frac{FP}{FP + TN}$

Для production: если класс «птица» редкий (1% выборки), accuracy (доля правильных ответов) будет 99% даже для тупого классификатора, всегда предсказывающего «не птица». Поэтому используем **F1-macro** (среднее F1 по всем классам) или **balanced accuracy**.

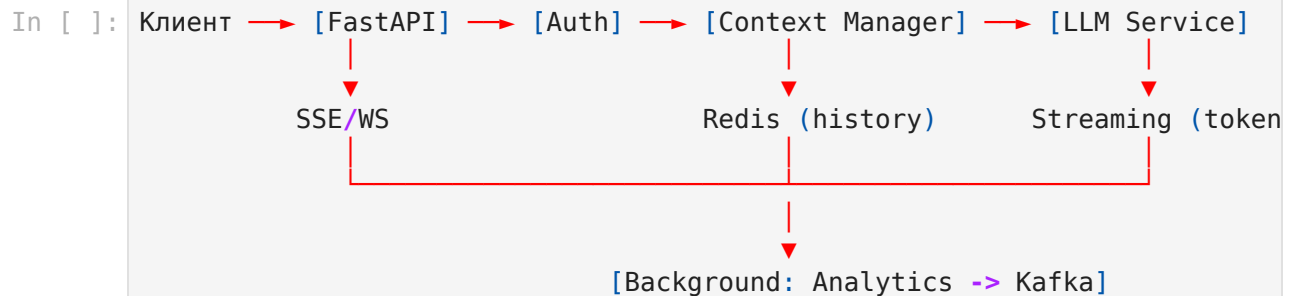
12.2. Кейс 2: Чат-бот с генеративной моделью

12.2.1. Постановка задачи и архитектура

Задача: чат-бот, который принимает сообщение пользователя, поддерживает историю диалога (контекст), генерирует ответ через LLM

(например, локальную модель через llama.cpp или внешний API OpenAI) и возвращает ответ **потоково** — токен за токеном. Также: фоновое логирование диалогов для аналитики.

Архитектура:



12.2.2. Управление контекстом диалога

LLM имеет ограниченный **context window** (например, 4096 токенов). Нужно хранить историю, но не превышать лимит.

In []:

```
from pydantic import BaseModel
from typing import List
import tiktoken # токенизатор OpenAI

class Message(BaseModel):
    role: str # "system", "user", "assistant"
    content: str

class ConversationManager:
    def __init__(self, model: str = "gpt-4", max_tokens: int = 4096):
        self.encoding = tiktoken.encoding_for_model(model)
        self.max_tokens = max_tokens
        self.reserve_tokens = 500 # запас для ответа модели

    def count_tokens(self, messages: List[Message]) -> int:
        # tiktoken считает токены точно
        return sum(len(self.encoding.encode(m.content)) for m in messages)

    def trim_context(self, messages: List[Message]) -> List[Message]:
        """Удаляет старые сообщения, пока не влезем в лимит."""
        # Системный промпт всегда оставляем (первое сообщение)
        system_msg = [m for m in messages if m.role == "system"]
        other_msgs = [m for m in messages if m.role != "system"]

        while True:
            total = self.count_tokens(system_msg + other_msgs)
            if total <= self.max_tokens - self.reserve_tokens:
                break
            if len(other_msgs) <= 1:
```



```

        break # оставляем минимум
other_msgs.pop(0) # удаляем самое старое

return system_msg + other_msgs

```

Почему `reserve_tokens` ? Модель генерирует ответ токен за токеном. Если контекст занимает 4096 токенов, а модель хочет сгенерировать ответ из 100 токенов — произойдёт ошибка. Нужно оставить «окно» для ответа.

Хранение в Redis:

```

In [ ]: from redis.asyncio import Redis
import json

async def get_conversation(
    user_id: str,
    conversation_id: str,
    redis: Redis,
    manager: ConversationManager,
) -> List[Message]:
    key = f"chat:{user_id}:{conversation_id}"
    data = await redis.lrange(key, 0, -1)

    messages = [Message(**json.loads(m)) for m in data]
    return manager.trim_context(messages)

async def add_message(
    user_id: str,
    conversation_id: str,
    message: Message,
    redis: Redis,
    ttl: int = 86400, # 24 часа
):
    key = f"chat:{user_id}:{conversation_id}"
    await redis.rpush(key, message.model_dump_json())
    await redis.expire(key, ttl)

```

12.2.3. Streaming через SSE: token-by-token

```

In [ ]: from fastapi import FastAPI
from fastapi.responses import StreamingResponse
import json
import asyncio

app = FastAPI()

async def generate_tokens_stream(
    messages: List[Message],
    model_client, # клиент к LLM (OpenAI, llama.cpp, etc.)
):

```

```

"""Асинхронный генератор токенов."""
# Предполагаем, что model_client.stream возвращает async iterator
async for token in model_client.stream_chat(messages):
    # SSE формат: data: {...}\n\n
    yield f"data: {json.dumps({'token': token})}\n\n"

# Финальное событие
yield f"data: {json.dumps({'done': True})}\n\n"

@app.post("/chat/stream")
async def chat_stream(
    request: ChatRequest,
    redis: Redis = Depends(get_redis),
):
    # Получаем/создаём контекст
    messages = await get_conversation(
        request.user_id, request.conversation_id, redis, conversation_man
    )
    messages.append(Message(role="user", content=request.message))

    # Сохраняем сообщение пользователя
    await add_message(request.user_id, request.conversation_id, messages[

    # StreamingResponse с media_type text/event-stream
    return StreamingResponse(
        generate_tokens_stream(messages, llm_client),
        media_type="text/event-stream",
        headers={
            "Cache-Control": "no-cache",
            "Connection": "keep-alive",
        },
    )

```

Клиент (JavaScript) получает:

```

In [ ]: const eventSource = new EventSource('/chat/stream');
eventSource.onmessage = (event) => {
    const data = JSON.parse(event.data);
    if (data.done) {
        eventSource.close();
    } else {
        appendTokenToUI(data.token);
    }
};

```

Почему SSE, а не WebSocket? Для одностороннего потока (сервер -> клиент) SSE проще: работает поверх HTTP, проходит через прокси, имеет автоматический reconnect. WebSocket нужен, если клиент тоже шлёт поток (например, голосовой ввод).

12.2.4. Ограничение токенов и стоимости

Если используется платный API (OpenAI, Anthropic), каждый токен стоит денег. Нужны лимиты.

```
In [ ]: from dataclasses import dataclass
import time

@dataclass
class UserQuota:
    daily_limit: int = 10000 # токенов в день
    used_today: int = 0
    reset_at: float = 0.0

class QuotaManager:
    def __init__(self, redis: Redis):
        self._redis = redis

    async def check_and_consume(self, user_id: str, requested_tokens: int) -> bool:
        key = f"quota:{user_id}"

        # Атомарный инкремент и проверка через Lua-скрипт
        # или pipeline
        pipe = self._redis.pipeline()
        pipe.get(key)
        pipe.ttl(key)
        result = await pipe.execute()

        used = int(result[0] or 0)
        ttl = result[1]

        if used + requested_tokens > 10000:
            return False

        # Если ключа нет – устанавливаем с TTL до конца дня
        if ttl == -1 or ttl == -2:
            seconds_until_midnight = self._seconds_to_midnight()
            await self._redis.setex(key, seconds_until_midnight, used + requested_tokens)
        else:
            await self._redis.incrby(key, requested_tokens)

        return True

    def _seconds_to_midnight(self) -> int:
        now = datetime.now()
        midnight = datetime.combine(now.date() + timedelta(days=1), time.time())
        return int((midnight - now).total_seconds())
```

На уровне модели: `max_tokens` параметр в API-запросе ограничивает длину ответа. Это защита от «runaway generation», когда модель начинает бесконечно повторять себя.

12.2.5. Фоновая обработка аналитики

После завершения диалога нужно: сохранить полный диалог в БД, отправить метрики, обновить агрегаты.

```
In [ ]: from fastapi import BackgroundTasks

@app.post("/chat")
async def chat(
    request: ChatRequest,
    background_tasks: BackgroundTasks,
    redis: Redis = Depends(get_redis),
):
    # Синхронный ответ (или streaming — отдельный endpoint)
    response = await generate_response(request)

    # Фоновая аналитика — не блокирует ответ
    background_tasks.add_task(
        log_conversation,
        user_id=request.user_id,
        conversation_id=request.conversation_id,
        messages=await get_full_conversation(...),
        latency_ms=response.latency_ms,
        token_count=response.token_count,
    )

    return response

async def log_conversation(**kwargs):
    # Сохранение в ClickHouse / BigQuery для аналитики
    # Отправка метрик в Prometheus
    # Обновление агрегатов в Redis
    ...
```

Почему BackgroundTasks, а не Celery? Логирование — не критичная операция. Если сервер перезапустится и потеряет одно событие — не страшно. Для критичных операций (оплата) — Celery/arq (Модуль 9).

12.2.6. Математическая подоплека: сэмплирование в генеративных моделях

LLM выдаёт распределение вероятностей по словарю: $P(w_t | w_1, \dots, w_{t-1})$. Как выбрать следующий токен?

Greedy decoding: всегда выбираем $\arg\max_w P(w)$.

Детерминированно, но скучно (повторы, отсутствие разнообразия).

Temperature sampling: делим логиты на температуру T перед

softmax:

$$P_T(w) = \frac{e^{z_w / T}}{\sum_{w'} e^{z_{w'} / T}}$$

- $T \rightarrow 0$: распределение становится острым, стремится к greedy.
- $T = 1$: исходное распределение.
- $T > 1$: распределение размывается, выбор случайнее, «креативнее».

Top-k sampling: выбираем только из k токенов с наибольшей вероятностью, остальным — 0.

Top-p (nucleus) sampling: выбираем минимальный набор токенов, суммарная вероятность которых $\geq p$.

Для production: $T = 0.7$, $\text{top}_p = 0.9$ — компромисс между когерентностью и разнообразием.

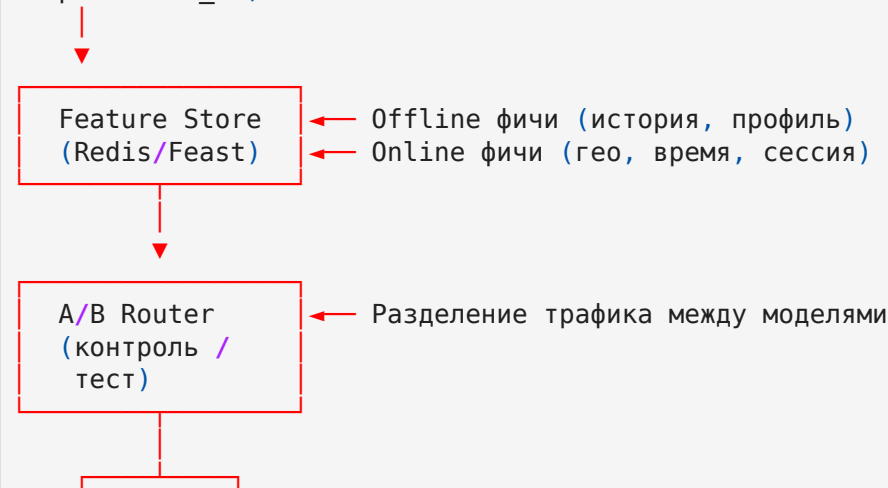
12.3. Кейс 3: Рекомендательная система

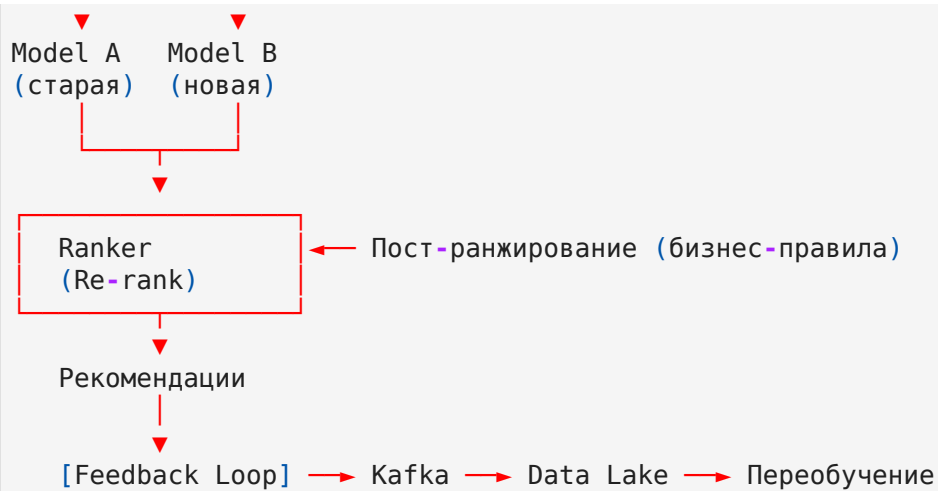
12.3.1. Постановка задачи и архитектура

Задача: сервис, который для пользователя и контекста (время, устройство, история) возвращает ранжированный список рекомендаций (товары, статьи, видео). Требования: latency < 50 мс, персонализация, A/B тестирование моделей.

Архитектура:

In []: Запрос: user_id, context





12.3.2. Feature retrieval: online vs offline

```

In [ ]: from feast import FeatureStore
import redis.asyncio as redis
from datetime import datetime

class FeatureRetriever:
    def __init__(self, feast_store: FeatureStore, redis_client: redis.Red
        self._feast = feast_store
        self._redis = redis_client

    async def get_features(self, user_id: str, context: dict) -> dict:
        features = {}

        # 1. Online фичи из Redis (< 5 мс)
        online = await self._redis.hgetall(f"user:{user_id}:features")
        features.update(online)

        # 2. Near-real-time фичи (счётчики, агрегаты за час)
        # Обновляются через Kafka Streams / Flink
        session_features = await self._redis.hgetall(f"session:{context['
        features.update(session_features)

        # 3. Offline фичи из Feast (предвычисленные, материализованные)
        # Вызываем синхронно, но Feast обычно fast (< 10 мс)
        feast_features = self._feast.get_online_features(
            features=[
                "user_features:age",
                "user_features:avg_purchase_30d",
                "user_features:category_affinity",
            ],
            entity_rows=[{"user_id": user_id}],
        ).to_dict()
        features.update(feast_features)

        # 4. Контекстные фичи (из запроса)
        features["hour_of_day"] = datetime.now().hour
  
```

```

features["day_of_week"] = datetime.now().weekday()
features["device_type"] = context.get("device", "unknown")

return features

```

Почему Redis для online, а Feast для offline? Feast материализует offline-фичи в низколатентное хранилище (Redis/DynamoDB), но его API синхронный и тяжелее. Прямое обращение к Redis для простых key-value быстрее. Гибридный подход даёт оптимальный баланс скорости и богатства фичей.

12.3.3. A/B testing: разделение трафика

```

In [ ]: import hashlib
        from enum import Enum

        class ModelVariant(Enum):
            CONTROL = "control"
            TREATMENT = "treatment_v2"

        def assign_variant(user_id: str, experiment_id: str, split: float = 0.5)
            """
            Детерминированное разделение на основе хеша.
            Один и тот же пользователь всегда попадает в ту же группу.
            """
            # Соль = experiment_id, чтобы разные эксперименты не коррелировали
            hash_input = f"{user_id}:{experiment_id}"
            hash_value = int(hashlib.md5(hash_input.encode()).hexdigest(), 16)

            # Нормализуем к [0, 1)
            normalized = hash_value / (2**128 - 1)

            if normalized < split:
                return ModelVariant.TREATMENT
            return ModelVariant.CONTROL

        @app.post("/recommend")
        async def recommend(
            request: RecommendRequest,
            redis: Redis = Depends(get_redis),
        ):
            variant = assign_variant(request.user_id, "rec_model_v2", split=0.1)

            features = await feature_retriever.get_features(request.user_id, request)

            if variant == ModelVariant.TREATMENT:
                recommendations = await model_v2.predict(features)
            else:
                recommendations = await model_v1.predict(features)

            # Логируем вариант для аналитики

```

```

return {
  "items": recommendations,
  "_experiment": {
    "id": "rec_model_v2",
    "variant": variant.value,
  }
}

```

Почему хеш, а не random? Если использовать `random.choice`, один и тот же пользователь в разные визиты может попасть в разные группы. Это искажает результаты (пользователь видит разные рекомендации и ведёт себя иначе). Хеширование гарантирует **стабильность** приписывания.

12.3.4. Метрики A/B теста

Что измеряем:

Метрика	Тип	Что показывает
CTR (Click-Through Rate)	Бинарная	Доля кликов по рекомендациям
Conversion Rate	Бинарная	Доля покупок после клика
Revenue per User	Непрерывная	Средний доход на пользователя
Dwell Time	Непрерывная	Время взаимодействия

Математическая подоплека: Z-тест для пропорций

Для CTR: пусть в группе A n_A пользователей, x_A кликов. В группе B: n_B , x_B .

$$\hat{p}_A = \frac{x_A}{n_A}, \quad \hat{p}_B = \frac{x_B}{n_B} \\ \hat{p} = \frac{x_A + x_B}{n_A + n_B} \quad \text{(объединённая пропорция)}$$

Статистика Z:

$$Z = \frac{\hat{p}_A - \hat{p}_B}{\sqrt{\hat{p}(1-\hat{p})\left(\frac{1}{n_A} + \frac{1}{n_B}\right)}}$$

При $|Z| > Z_{\alpha/2}$ (1.96 для $\alpha = 0.05$) различие статистически значимо.

Minimum Detectable Effect (MDE): минимальный эффект, который мы способны обнаружить при заданной мощности (обычно 80%) и уровне

значимости (5%).

$$\frac{2 \cdot \sigma^2 \cdot (Z_{1-\alpha/2} + Z_{1-\beta})^2}{\text{MDE}^2}$$

Для CTR = 5%, MDE = 0.5% (относительный +10%), $\alpha = 0.05$, $\beta = 0.2$:

$$\frac{2 \cdot 0.05 \cdot 0.95 \cdot (1.96 + 0.84)^2}{0.005^2} \approx 29,960 \text{ на группу}$$

Практический вывод: для обнаружения маленьких улучшений нужны десятки тысяч пользователей. Нельзя судить об A/B тесте по 100 пользователям.

12.3.5. Feedback loop: логирование и обучение

```
In [ ]: async def log_interaction(
    user_id: str,
    item_id: str,
    experiment_id: str,
    variant: str,
    rank: int, # позиция в выдаче
    event_type: str, # "impression", "click", "purchase"
    redis: Redis,
):
    """Логирует взаимодействие для переобучения модели."""
    event = {
        "timestamp": datetime.utcnow().isoformat(),
        "user_id": user_id,
        "item_id": item_id,
        "experiment": experiment_id,
        "variant": variant,
        "rank": rank,
        "event": event_type,
    }

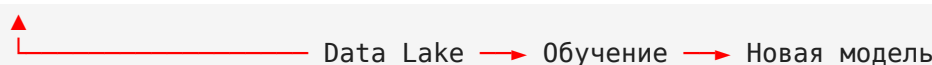
    # 1. В Kafka для real-time processing
    await kafka_producer.send("user_interactions", json.dumps(event).encode())

    # 2. В Redis для real-time агрегатов (счётчики)
    if event_type == "click":
        await redis.hincrby(f"user:{user_id}:clicks", item_id, 1)

    # 3. В БД для offline анализа
    # (через outbox pattern для надёжности)
```

Feedback loop замкнут:

```
In [ ]: Рекомендации → Пользователь → Клик/Покупка → Kafka →
```



Проблема *delayed feedback*: пользователь может кликнуть через час после показа. Модель уже обучилась на устаревших данных. Решение: использовать **importance weighting** или обучаться на логах с задержкой.

12.4. Финальный проект: полноценный асинхронный бэкенд для ML-сервиса

12.4.1. Требования и архитектура

Продукт: ML-платформа для автоматической классификации документов.

Функциональные требования:

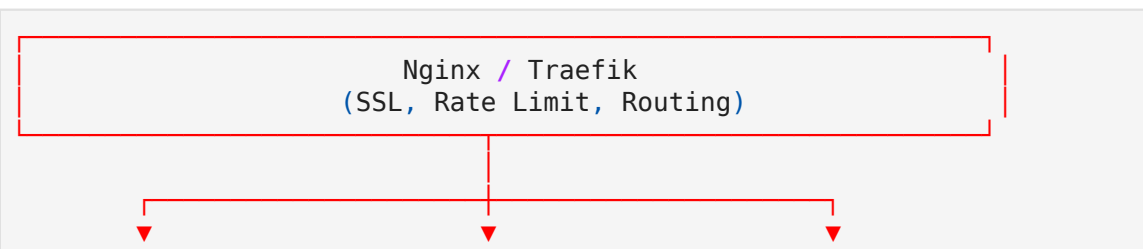
1. Регистрация/аутентификация пользователей (JWT).
2. Загрузка документа (PDF/изображение) -> классификация.
3. История предсказаний с фильтрацией и пагинацией.
4. Асинхронная обработка тяжёлых документов (> 10 страниц) через очередь.
5. WebSocket-уведомления о завершении обработки.
6. Экспорт истории в CSV.
7. A/B тестирование моделей (v1 vs v2).

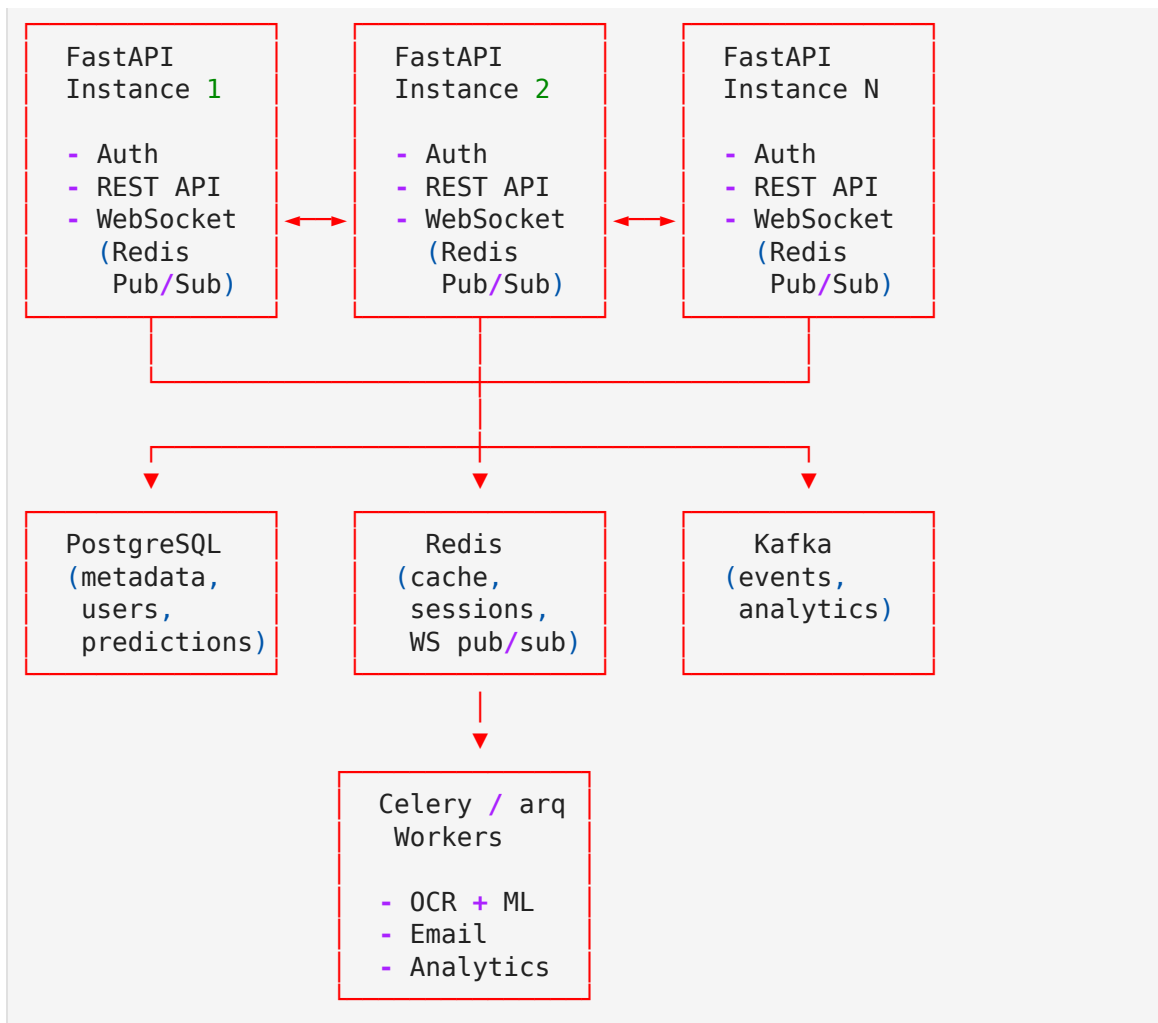
Нефункциональные требования:

- Latency p95 < 300 мс для синхронных запросов.
- Доступность 99.9%.
- Горизонтальное масштабирование.
- Структурированные логи, метрики Prometheus, трейсы OpenTelemetry.

Архитектура (высокоуровневая):

In []:





12.4.2. Структура проекта (Clean Architecture)

```

In [ ]: document_classifier/
├── pyproject.toml
├── docker-compose.yml
├── Dockerfile
├── alembic/
│   └── versions/
├── app/
│   ├── __init__.py
│   ├── main.py           # FastAPI app, lifespan
│   ├── config.py         # Pydantic Settings (Module 10)
│   └── domain/
│       ├── __init__.py
│       ├── entities.py   # User, Document, Prediction
│       ├── repositories.py # Protocols
│       └── exceptions.py  # Domain exceptions
├── application/
│   ├── __init__.py
│   ├── auth_use_cases.py
│   └── predict_use_case.py

```

```

├── export_use_case.py
├── infrastructure/
│   ├── __init__.py
│   ├── db/
│   │   ├── connection.py    # AsyncEngine, sessionmaker
│   │   ├── models.py        # SQLAlchemy ORM
│   │   └── repositories.py   # Реализации Protocol
│   ├── cache/
│   │   └── redis_client.py
│   ├── ml/
│   │   ├── model_loader.py   # ONNX / Torch
│   │   └── inference.py
│   ├── messaging/
│   │   ├── kafka_producer.py
│   │   └── celery_tasks.py
│   ├── security/
│   │   ├── password.py       # bcrypt
│   │   └── jwt.py            # encode/decode
├── presentation/
│   ├── __init__.py
│   ├── dependencies.py       # DI (Module 5)
│   ├── routers/
│   │   ├── auth.py
│   │   ├── predictions.py
│   │   ├── websocket.py
│   │   └── admin.py
│   ├── middleware/
│   │   ├── correlation.py     # Correlation ID
│   │   ├── timing.py          # Request timing
│   │   └── exception_handler.py
├── tests/
│   ├── conftest.py
│   ├── unit/
│   ├── integration/
│   └── e2e/
├── scripts/
└── migrate.sh

```

12.4.3. Аутентификация и авторизация

```

In [ ]: # presentation/routers/auth.py
from fastapi import APIRouter, Depends, HTTPException
from fastapi.security import OAuth2PasswordBearer, OAuth2PasswordRequestF
from application.auth_use_cases import RegisterUseCase, LoginUseCase
from domain.entities import UserCreate, Token

router = APIRouter(prefix="/auth", tags=["auth"])
oauth2_scheme = OAuth2PasswordBearer(tokenUrl="/auth/login")

@router.post("/register", response_model=UserRead)
async def register(
    data: UserCreate,
    use_case: RegisterUseCase = Depends(get_register_use_case),

```

```

):
    return await use_case.execute(data)

@router.post("/login", response_model=Token)
async def login(
    form_data: OAuth2PasswordRequestForm = Depends(),
    use_case: LoginUseCase = Depends(get_login_use_case),
):
    return await use_case.execute(form_data.username, form_data.password)

# Зависимость для защиты endpoint'ов
async def get_current_user(
    token: str = Depends(oauth2_scheme),
    jwt_service: JWTService = Depends(get_jwt_service),
) -> User:
    payload = jwt_service.decode(token)
    if not payload:
        raise HTTPException(401, "Invalid token")
    return User(id=payload["sub"], email=payload["email"])

@router.get("/me", response_model=UserRead)
async def me(current_user: User = Depends(get_current_user)):
    return current_user

```

12.4.4. База данных и миграции

```

In [ ]: # infrastructure/db/connection.py
from sqlalchemy.ext.asyncio import create_async_engine, async_sessionmaker
from sqlalchemy.orm import declarative_base

Base = declarative_base()

engine = create_async_engine(
    settings.database_url,
    pool_size=20,
    max_overflow=10,
    pool_pre_ping=True, # проверка соединения перед использованием
)

async_session_maker = async_sessionmaker(
    engine,
    class_=AsyncSession,
    expire_on_commit=False,
)

# Зависимость для DI
async def get_db() -> AsyncSession:
    async with async_session_maker() as session:
        yield session

```

Alembic для миграций:

```
In [ ]: # Создание миграции
alembic revision --autogenerate -m "add predictions table"

# Применение
alembic upgrade head

# В CI/CD (Module 10)
# Миграции применяются ДО деплоя новой версии
```

12.4.5. ML-интеграция с ProcessPoolExecutor

```
In [ ]: # infrastructure/ml/inference.py
from concurrent.futures import ProcessPoolExecutor
import asyncio
import numpy as np

# Пул процессов для CPU-bound OCR + inference
_ml_executor = ProcessPoolExecutor(
    max_workers=4,
    initializer=_init_worker,
)

def _init_worker():
    """Загрузка модели в каждом воркере один раз."""
    global _model
    _model = DocumentClassifierModel.load("/models/doc_classifier.onnx")

def _predict_sync(image_bytes: bytes) -> dict:
    """Синхронная функция, выполняется в отдельном процессе."""
    # OCR + предобработка + ONNX inference
    # GIL не мешает – отдельный процесс
    return _model.predict(image_bytes)

async def predict_document(image_bytes: bytes) -> dict:
    loop = asyncio.get_running_loop()
    return await loop.run_in_executor(_ml_executor, _predict_sync, image_
```

12.4.6. Фоновые задачи через arq

```
In [ ]: # infrastructure/messaging/tasks.py
from arq import create_pool, Actor
from arq.connections import RedisSettings

async def heavy_ocr_task(ctx, document_id: str, file_url: str):
    """Асинхронная задача для тяжёлых документов."""
    # Скачивание, OCR, inference
    # Может занимать минуты
    result = await process_large_document(file_url)

    # Уведомление через WebSocket (Redis Pub/Sub)
    await ctx['redis'].publish(
```

```

        f"ws:{document_id}",
        json.dumps({"status": "completed", "result": result})
    )

    return result

class WorkerSettings:
    redis_settings = RedisSettings(host="redis")
    functions = [heavy_ocr_task]
    max_jobs = 10

# B FastAPI
@app.post("/documents/async")
async def upload_async(
    file: UploadFile,
    background: BackgroundTasks,
    redis: Redis = Depends(get_redis),
):
    # Сохраняем файл, получаем ID
    doc_id = await save_upload(file)

    # Ставим в очередь
    job = await arq_redis.enqueue_job(
        'heavy_ocr_task',
        doc_id,
        file_url,
    )

    return {"document_id": doc_id, "job_id": job.job_id, "status": "queue"}

```

12.4.7. WebSocket-уведомления

```

In [ ]: # presentation/routers/websocket.py
from fastapi import WebSocket, WebSocketDisconnect
import json

@app.websocket("/ws/notifications/{user_id}")
async def notifications(websocket: WebSocket, user_id: str):
    await websocket.accept()

    # Подписка на Redis Pub/Sub для этого пользователя
    pubsub = redis_client.pubsub()
    await pubsub.subscribe(f"ws:user:{user_id}")

    try:
        async for message in pubsub.listen():
            if message["type"] == "message":
                await websocket.send_text(message["data"].decode())
    except WebSocketDisconnect:
        await pubsub.unsubscribe()

```

12.4.8. Тестирование

Unit-тесты (Domain Layer):

```
In [ ]: # tests/unit/test_predict_use_case.py
import pytest
from unittest.mock import AsyncMock
from application.predict_use_case import PredictUseCase
from domain.entities import Prediction

@pytest.mark.asyncio
async def test_predict_use_case_success():
    mock_repo = AsyncMock()
    mock_ml = AsyncMock()
    mock_ml.predict.return_value = 0.95

    use_case = PredictUseCase(mock_repo, mock_ml)
    result = await use_case.execute("user_1", b"fake_image")

    assert isinstance(result, Prediction)
    assert result.confidence == 0.95
    mock_repo.save.assert_awaited_once()
```

Интеграционные тесты:

```
In [ ]: # tests/integration/test_predictions.py
import pytest
from httpx import AsyncClient

@pytest.mark.asyncio
async def test_create_prediction(client: AsyncClient, auth_headers: dict):
    response = await client.post(
        "/predictions",
        files={"file": ("test.jpg", b"fake_jpg_bytes", "image/jpeg")},
        headers=auth_headers,
    )
    assert response.status_code == 201
    assert "id" in response.json()
```

Нагрузочное тестирование:

```
In [ ]: # locustfile.py
from locust import HttpUser, task, between

class MLUser(HttpUser):
    wait_time = between(1, 3)

    @task
    def predict(self):
        self.client.post(
            "/predictions",
```



```

        files={"file": open("test.jpg", "rb")},
        headers={"Authorization": "Bearer token"}
    )

```

```
In [ ]: locust -f locustfile.py --host http://localhost:8000
```

12.4.9. Метрики и observability

```
In [ ]: # presentation/middleware/timing.py
from prometheus_client import Histogram, Counter
import time

REQUEST_LATENCY = Histogram(
    "http_request_duration_seconds",
    "Request latency",
    ["method", "endpoint", "status"],
    buckets=[0.01, 0.025, 0.05, 0.1, 0.25, 0.5, 1.0, 2.5, 5.0],
)

REQUEST_COUNT = Counter(
    "http_requests_total",
    "Total requests",
    ["method", "endpoint", "status"],
)

@app.middleware("http")
async def prometheus_middleware(request: Request, call_next):
    start = time.time()
    response = await call_next(request)
    duration = time.time() - start

    REQUEST_LATENCY.labels(
        method=request.method,
        endpoint=request.url.path,
        status=response.status_code,
    ).observe(duration)

    REQUEST_COUNT.labels(
        method=request.method,
        endpoint=request.url.path,
        status=response.status_code,
    ).inc()

    return response

```

OpenTelemetry трейс:

```
In [ ]: from opentelemetry.instrumentation.fastapi import FastAPIInstrumentor
from opentelemetry.instrumentation.sqlalchemy import SQLAlchemyInstrumentor

FastAPIInstrumentor.instrument_app(app)
SQLAlchemyInstrumentor().instrument()
```

12.4.10. Деплой и CI/CD

Dockerfile (multi-stage):

```
In [ ]: FROM python:3.12-slim AS builder
WORKDIR /app
RUN pip install uv
COPY pyproject.toml .
RUN uv pip install --system -e .

FROM python:3.12-slim AS runtime
WORKDIR /app
COPY --from=builder /usr/local/lib/python3.12/site-packages /usr/local/li
COPY --from=builder /usr/local/bin /usr/local/bin
COPY ./app ./app
USER appuser
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

GitHub Actions:

```
In [ ]: name: CI/CD
on:
  push:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    services:
      postgres:
        image: postgres:16-alpine
        env:
          POSTGRES_USER: test
          POSTGRES_PASSWORD: test
          POSTGRES_DB: test_db
        ports: ["5432:5432"]
      redis:
        image: redis:7-alpine
        ports: ["6379:6379"]
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: "3.12" }
      - run: pip install uv && uv pip install --system -e ".[dev]"
      - run: alembic upgrade head
      - run: pytest --cov=app --cov-report=xml
      - run: ruff check . && mypy app/
```

12.4.11. Код-ревью и оптимизация

Чек-лист код-ревью для ML-бэкенда:

Аспект	Что проверять
Безопасность	Нет ли <code>eval()</code> , <code>exec()</code> ? Валидация входных файлов? SQL-инъекции?
Производительность	Нет ли синхронных блокировок в <code>async def</code> ? Используется ли <code>run_in_executor</code> для CPU-bound?
Надёжность	Есть ли Circuit Breaker для внешних сервисов? Retry с backoff? Idempotency?
Observability	Есть ли structured logs, correlation ID, метрики для всех критичных путей?
Чистота	Domain Layer зависит ли от Infrastructure? Правильно ли используются Protocol/ABC?
ML-специфика	Предобработка идентична обучению? Есть ли fallback при отказе модели?

Профилирование:

```
In [ ]: # CPU профиль
py-spy top --pid $(pgrep -f uvicorn)

# Память
memray run -m uvicorn app.main:app

# Asyncio debug
PYTHONASYNCIODEBUG=1 uvicorn app.main:app
```

Итог модуля 12

Кейс / Компонент	Ключевые технологии	Интегрированные модули
Кейс 1: Классификация	ONNX Runtime, Redis cache, Token Bucket, SHA-256	4 (Pydantic), 6 (Redis), 7 (Rate limit), 9 (ONNX), 10 (Docker)
Кейс 2: Чат-бот	SSE streaming, tiktoken, temperature sampling, BackgroundTasks	2 (async generators), 4 (StreamingResponse), 9 (фоновые задачи), 11 (CQRS)
Кейс 3: Рекомендации	Feature Store, A/B testing, Z-тест, Feedback loop	5 (DI), 6 (Feast/Redis), 9 (Feature Store), 11 (Event-Driven)
Финальный проект	Clean Architecture, JWT, SQLAlchemy 2.0, arq, WebSocket, Prometheus, OpenTelemetry	Все модули 1-11

Кейс / Компонент	Ключевые технологии	Интегрированные модули
Код-ревью	py-spy, memray, asyncio debug, чек-лист	8 (тесты), 10 (observability), 11 (паттерны)

Что вы умеете после курса:

- Проектировать асинхронные системы от первых принципов (итераторы -> корутины -> event loop).
- Строить production-ready FastAPI-приложения с валидацией, DI, чистой архитектурой.
- Интегрировать ML-модели без блокировки event loop.
- Обеспечивать безопасность, observability и масштабируемость.
- Выбирать между REST, gRPC и Event-Driven под задачу.
- Проводить A/B тесты и интерпретировать результаты статистически.

Курс завершён. Дальше — практика, код-ревью, профилирование реальных систем и постоянное углубление в каждый из модулей по мере встречи с новыми задачами.